

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Криптография и сетевая безопасность»

Выполнил:
Студент 4-го курса
Группы АС-66
Головкина В.Д.
Проверил:
Хацкевич А.С.

Цель работы: приобретение практических навыков кодирования и декодирования двоичных данных при использовании кода Хемминга и при использовании итеративных кодов. Вариант 1

Ход работы

Задание 1-2: Разработать приложение для кодирования/декодирования двоичной информации кодом Хэмминга с минимальным кодовым расстоянием 3 или 4. Разработать приложение для кодирования/декодирования двоичной информации итеративным кодом с различной относительной избыточностью кодовых слов.

Код:

```
import random

def fact(n):
    res = 1
    for i in range(2, n + 1):
        res *= i
    return res

def binom(wt, r):
    if wt < 0 or wt > r:
        return 0
    return fact(r) // (fact(wt) * fact(r - wt))

def int_to_bits(num, width):
    return [int(c) for c in bin(num)[2:].zfill(width)]

def wt_of(num):
    return bin(num).count('1')

def bits_to_str(bits):
    return ''.join(str(b) for b in bits)

def text_to_bits(text):
    bits = []
    for ch in text:
        bits.extend(int(b) for b in bin(ord(ch))[2:].zfill(8))
    return bits

def bits_to_text(bits):
    chars = []
    for i in range(0, len(bits) - 7, 8):
        code = 0
        for b in bits[i:i + 8]:
            code = (code << 1) | b
        chars.append(chr(code))
```

```

    return "".join(chars)
def chunk_bits(bits, k):
    out = []
    for i in range(0, len(bits), k):
        block = bits[i:i + k]
        if len(block) < k:
            block += [0] * (k - len(block))
        out.append(block)
    return out
def input_message():
    print("\nВвод информационного сообщения:")
    print("1. Текст (ASCII)")
    print("2. Двоичная строка")
    print("3. Файл")
    choice = input("Выбор: ").strip()

    if choice == '1':
        s = input("Введите текст: ")
        bits = text_to_bits(s)
    elif choice == '2':
        s = input("Введите строку из 0 и 1: ").strip()
        bits = [int(c) for c in s if c in '01']
    elif choice == '3':
        path = input("Путь к файлу: ").strip()
        try:
            with open(path, 'r', encoding='utf-8') as f:
                bits = text_to_bits(f.read())
        except OSError:
            print("Не удалось открыть файл.")
            return None
    else:
        print("Неверный выбор.")
        return None

    if len(bits) < 16:
        print(f"Длина ({len(bits)}) < 16. Требуется не менее 16 бит.")
        return None

    print(f"Получено бит: {len(bits)}")
    print(f"Биты: {bits_to_str(bits)}")

```

```

    return bits
def add_errors(word, count):
    if count == 0:
        return list(word), []
    count = min(count, len(word))
    positions = random.sample(range(len(word)), count)
    corrupted = list(word)
    for p in positions:
        corrupted[p] ^= 1
    return corrupted, positions
class HammingCode:
    def __init__(self, k, dmin=3):
        if k < 1:
            raise ValueError("k >= 1")
        if dmin not in (3, 4):
            raise ValueError("dmin = 3 или 4")

        self.k = k
        self.dmin = dmin

        r = 1
        while (1 << r) < k + r + 1:
            r += 1
        if dmin == 4:
            r += 1
        self.r = r
        self.r_p = r - 1 if dmin == 4 else r
        self.n = k + r

        self.H = self._build_H()
        self.G = self._build_G()
        self.syndrome_table = self._build_syndrome_table()

    def _build_H(self):
        k, r, r_p = self.k, self.r, self.r_p
        H = [[0] * self.n for _ in range(r)]

        cols = []
        wt = 2
        num = 3

```

```

while len(cols) < k:
    total = binom(wt, r_p)
    found = 0
    while found < total and len(cols) < k:
        if wt_of(num) == wt and num < (1 << r_p):
            cols.append(int_to_bits(num, r_p))
            found += 1
        num += 1
        if num >= (1 << r_p):
            break
    wt += 1

```

```

for j, col in enumerate(cols):
    for i in range(r_p):
        H[i][j] = col[i]

```

```

for j in range(k, k + r_p):
    H[j - k][j] = 1

```

```

if self.dmin == 4:
    for j in range(self.n):
        H[r - 1][j] = 1

```

```

return H

```

```

def _build_G(self):
    k, r_p = self.k, self.r_p
    P = [[self.H[i][j] for i in range(r_p)] for j in range(k)]

```

```

G = [[0] * self.n for _ in range(k)]
for i in range(k):
    G[i][i] = 1
    for j in range(r_p):
        G[i][k + j] = P[i][j]

```

```

if self.dmin == 4:
    for i in range(k):
        G[i][self.n - 1] = 1
return G

```

```

def _build_syndrome_table(self):
    table = {}
    for pos in range(self.n):
        syn = tuple(self.H[i][pos] for i in range(self.r))
        if any(syn):
            table[syn] = pos
    return table

def encode(self, msg):
    return [sum(msg[i] & self.G[i][j] for i in range(self.k)) % 2
            for j in range(self.n)]

def syndrome(self, recv):
    return [sum(self.H[i][j] & recv[j] for j in range(self.n)) % 2
            for i in range(self.r)]

def recalc_parity(self, Yk):
    Yr_prime = []
    for i in range(self.r_p):
        s = 0
        for j in range(self.k):
            s ^= (self.H[i][j] & Yk[j])
        Yr_prime.append(s)
    if self.dmin == 4:
        Yr_prime.append(sum(Yk) % 2)
    return Yr_prime

def decode(self, recv):
    S = self.syndrome(recv)
    E = [0] * self.n

    if all(b == 0 for b in S):
        return list(recv), S, E, 'ok'

    if self.dmin == 3:
        pos = self.syndrome_table.get(tuple(S))
        if pos is not None:
            E[pos] = 1
            corrected = [recv[i] ^ E[i] for i in range(self.n)]
            return corrected, S, E, 'corrected'

```

```

    return list(recv), S, E, 'uncorrectable'

if S[-1] == 1 and any(S[:-1]):
    pos = self.syndrome_table.get(tuple(S[:-1]))
    if pos is not None and pos < self.n - 1:
        E[pos] = 1
        corrected = [recv[i] ^ E[i] for i in range(self.n)]
        return corrected, S, E, 'corrected'
    return list(recv), S, E, 'uncorrectable'
if S[-1] == 1 and not any(S[:-1]):
    E[self.n - 1] = 1
    corrected = [recv[i] ^ E[i] for i in range(self.n)]
    return corrected, S, E, 'corrected'
if S[-1] == 0 and any(S[:-1]):
    return list(recv), S, E, 'detected'
return list(recv), S, E, 'uncorrectable'

```

```

class IterativeCode:

```

```

    def __init__(self, k1, k2, groups=2):
        if k1 < 2 or k2 < 2:
            raise ValueError("k1, k2 >= 2")
        if groups not in (2, 3, 4):
            raise ValueError("groups = 2, 3 или 4")

        self.k1 = k1
        self.k2 = k2
        self.k = k1 * k2
        self.groups = groups
        self.r_h = k1
        self.r_v = k2
        self.r_hv = 1 if groups >= 3 else 0
        self.r_d = (k1 + k2 - 1) * 2 if groups == 4 else 0

        self.r = self.r_h + self.r_v + self.r_hv + self.r_d
        self.n = self.k + self.r

```

```

    def encode(self, Xk):
        M = [Xk[i * self.k2:(i + 1) * self.k2] for i in range(self.k1)]

```

```
Xh = [sum(row) % 2 for row in M]
Xv = [sum(M[i][j] for i in range(self.k1)) % 2 for j in range(self.k2)]
```

```
result = []
for row in M:
    result.extend(row)
result.extend(Xh)
result.extend(Xv)

if self.groups >= 3:
    result.append((sum(Xh) + sum(Xv)) % 2)

if self.groups == 4:
    result.extend(self._diag_parities(M))

return result
```

```
def _diag_parities(self, M):
    k1, k2 = self.k1, self.k2
    diag = []

    for d in range(-(k2 - 1), k1):
        s = 0
        cnt = 0
        for i in range(k1):
            j = i - d
            if 0 <= j < k2:
                s ^= M[i][j]
                cnt += 1
        if cnt > 1:
            diag.append(s)

    for d in range(1, k1 + k2 - 2):
        s = 0
        cnt = 0
        for i in range(k1):
            j = d - i
            if 0 <= j < k2:
                s ^= M[i][j]
                cnt += 1
```

```
if cnt > 1:  
    diag.append(s)
```

```
return diag
```

```
def _extract_matrix(self, Yn):  
    return [Yn[i * self.k2:(i + 1) * self.k2] for i in range(self.k1)]
```

```
def decode(self, Yn):  
    M = [row[:] for row in self._extract_matrix(Yn)]  
    Xh_recv = Yn[self.k:self.k + self.r_h]  
    Xv_recv = Yn[self.k + self.r_h:self.k + self.r_h + self.r_v]  
    Xh_calc = [sum(M[i]) % 2 for i in range(self.k1)]  
    Xv_calc = [sum(M[i][j] for i in range(self.k1)) % 2  
               for j in range(self.k2)]
```

```
row_err = [i for i in range(self.k1) if Xh_recv[i] != Xh_calc[i]]  
col_err = [j for j in range(self.k2) if Xv_recv[j] != Xv_calc[j]]
```

```
errors = [(i, j) for i in row_err for j in col_err]
```

```
Xhv_ok = True  
if self.groups >= 3:  
    Xhv_recv = Yn[self.k + self.r_h + self.r_v]  
    Xhv_calc = (sum(Xh_recv) + sum(Xv_recv)) % 2  
    Xhv_ok = (Xhv_recv == Xhv_calc)
```

```
corrected = [row[:] for row in M]  
for (i, j) in errors:  
    corrected[i][j] ^= 1
```

```
Yk_corr = []  
for row in corrected:  
    Yk_corr.extend(row)
```

```
if not row_err and not col_err and Xhv_ok:  
    status = 'ok'  
elif errors and len(row_err) == len(col_err):  
    status = 'corrected'  
elif row_err or col_err:
```

```

        status = 'detected'
    else:
        status = 'uncorrectable'

    info = {
        'row_parity_errors': row_err,
        'col_parity_errors': col_err,
        'errors_found': errors,
        'Xhv_ok': Xhv_ok,
        'status': status,
    }
    return Yk_corr, info

```

```

def run_hamming():
    print("\n" + "=" * 60)
    print("  ЛАБА 2, ЧАСТЬ 1: КОД ХЕММИНГА")
    print("=" * 60)

    try:
        k = int(input("Введите k (длина информационного блока, >= 1): ").strip())
    except ValueError:
        print("Некорректное k.")
        return

    try:
        dmin = int(input("Введите dmin (3 или 4): ").strip())
    except ValueError:
        print("Некорректный dmin.")
        return

    try:
        code = HammingCode(k, dmin)
    except ValueError as e:
        print(f"Ошибка: {e}")
        return

    print(f"\nПараметры кода: (n, k) = ({code.n}, {code.k}), "
          f"r = {code.r}, dmin = {code.dmin}")
    print(f"Скорость кода R = k/n = {code.k}/{code.n} = {code.k / code.n:.3f}")

```

```

print("\nПроверочная матрица H:")
for row in code.H:
    print(" " + " ".join(str(b) for b in row))
print("\nПорождающая матрица G = [I_k | P]:")
for row in code.G:
    print(" " + " ".join(str(b) for b in row))

bits = input_message()
if bits is None:
    return

blocks = chunk_bits(bits, code.k)
print(f"\nСообщение разбито на {len(blocks)} блок(ов) по {code.k} бит.")

for idx, block in enumerate(blocks):
    print("\n" + "-" * 60)
    print(f"Блок #{idx + 1}")
    print("-" * 60)

    print(f"Xk          : {bits_to_str(block)}")
    encoded = code.encode(block)
    print(f"Xn          : {bits_to_str(encoded)}")
    print(f"Xr          : {bits_to_str(encoded[code.k:])}")

s = input("Число ошибок (0, 1, 2): ").strip()
try:
    err_count = int(s)
except ValueError:
    err_count = 0

received, positions = add_errors(encoded, err_count)
if positions:
    print(f"Ошибки внесены в позиции: {positions}")
else:
    print("Ошибки не вносились.")
print(f"Yn          : {bits_to_str(received)}")

Yk = received[:code.k]
Yr = received[code.k:]

```

```
Yr_prime = code.recalc_parity(Yk)
S = [Yr[i] ^ Yr_prime[i] for i in range(len(Yr))]
```

```
print(f"Yr          : {bits_to_str(Yr)}")
print(f"Yr'         : {bits_to_str(Yr_prime)}")
print(f"S = Yr XOR Yr' : {bits_to_str(S)}")
```

```
corrected, S_full, E, status = code.decode(received)
print(f"En          : {bits_to_str(E)}")
print(f"Исправленное слово : {bits_to_str(corrected)}")
print(f"Статус       : {status}")
print(f"Xk верно      : {corrected[:code.k] == block}")
```

```
def run_iterative():
```

```
    print("\n" + "=" * 60)
    print("  ЛАБА 2, ЧАСТЬ 2: ИТЕРАТИВНЫЙ КОД")
    print("=" * 60)
```

```
    print("\nПримеры k1 x k2: 4x4, 4x5, 4x6, 4x8, 5x8")
```

```
    try:
```

```
        k1 = int(input("Введите k1 (строк, >= 2): ").strip())
        k2 = int(input("Введите k2 (столбцов, >= 2): ").strip())
```

```
    except ValueError:
```

```
        print("Некорректные k1, k2.")
        return
```

```
    try:
```

```
        groups = int(input("Число групп паритетов (2, 3 или 4): ").strip())
```

```
    except ValueError:
```

```
        groups = 2
```

```
    try:
```

```
        code = IterativeCode(k1, k2, groups)
```

```
    except ValueError as e:
```

```
        print(f"Ошибка: {e}")
        return
```

```
    print(f"\nПараметры кода: k = {code.k}, r = {code.r}, n = {code.n}, "
```

```

    f"групп паритетов = {code.groups}")
print(f"Скорость кода  $R = k/n = \{code.k\} / \{code.n\} = \{code.k / code.n:.3f\}$ ")

bits = input_message()
if bits is None:
    return

blocks = chunk_bits(bits, code.k)
print(f"\nСообщение разбито на {len(blocks)} блок(ов) по {code.k} бит.")

for idx, block in enumerate(blocks):
    print("\n" + "-" * 60)
    print(f"Блок #{idx + 1}")
    print("-" * 60)

    print(f"Xk          : {bits_to_str(block)}")
    encoded = code.encode(block)
    print(f"Xn          : {bits_to_str(encoded)}")
    print(f"Xr          : {bits_to_str(encoded[code.k:])}")

    print("Матрица k1 x k2:")
    for i in range(code.k1):
        row = block[i * code.k2:(i + 1) * code.k2]
        print("  " + bits_to_str(row))

s = input("Число ошибок: ").strip()
try:
    err_count = int(s)
except ValueError:
    err_count = 0

received, positions = add_errors(encoded, err_count)
if positions:
    print(f"Ошибки внесены в позиции: {positions}")
else:
    print("Ошибки не вносились.")
print(f"Yn          : {bits_to_str(received)}")

Yk_corr, info = code.decode(received)
print(f"\nСтроки с ошибкой паритета : {info['row_parity_errors']}")

```

```

print(f"Столбцы с ошибкой паритета: {info['col_parity_errors']}")
print(f"Найденные позиции ошибок : {info['errors_found']}")
print(f"Исправленное Yk'      : {bits_to_str(Yk_corr)}")
print(f"Статус                : {info['status']}")
print(f"Xk верно              : {Yk_corr == block}")

```

```
def run_analysis():
```

```

    print("\n" + "=" * 60)
    print(" АНАЛИЗ КОРРЕКТИРУЮЩЕЙ СПОСОБНОСТИ")
    print("=" * 60)

```

```
    try:
```

```

        k1 = int(input("Введите k1: ").strip())
        k2 = int(input("Введите k2: ").strip())
        groups = int(input("Число групп паритетов (2, 3, 4): ").strip())
        N1 = int(input("Сколько испытаний N1 на кратность? ").strip())
        max_err = int(input("Максимальная кратность ошибок: ").strip())

```

```
    except ValueError:
```

```

        print("Некорректные параметры.")
        return

```

```
    try:
```

```
        code = IterativeCode(k1, k2, groups)
```

```
    except ValueError as e:
```

```

        print(f"Ошибка: {e}")
        return

```

```
print(f"\nПараметры: k = {code.k}, n = {code.n}, групп = {code.groups}")
```

```
Xk = [random.randint(0, 1) for _ in range(code.k)]
```

```
Xn = code.encode(Xk)
```

```
print(f"\nЭталонное Xk = {bits_to_str(Xk)}")
```

```
print(f"Эталонное Xn = {bits_to_str(Xn)}")
```

```
print(f"\n{'i':>3} {'N2/N1':>10} {'N3/N1':>10}")
```

```
print("-" * 30)
```

```
for i in range(1, max_err + 1):
```

```
    N2 = 0
```

```
    N3 = 0
```

```

for _ in range(N1):
    Yn, _ = add_errors(Xn, i)
    Yk_corr, info = code.decode(Yn)
    if len(info['errors_found']) == i:
        N2 += 1
    if Yk_corr == Xk:
        N3 += 1
print(f"{i:>3} {N2 / N1:>10.3f} {N3 / N1:>10.3f}")
print("\nN2/N1 — доля верно определённой кратности")
print("N3/N1 — доля полностью исправленных ошибок")

```

```

def main():
    while True:
        print("\n" + "=" * 60)
        print(" ЛАБА 2. ПОМЕХОУСТОЙЧИВОЕ КОДИРОВАНИЕ")
        print("=" * 60)
        print("1. Часть 1: Код Хемминга")
        print("2. Часть 2: Итеративный код")
        print("3. Анализ корректирующей способности итеративного кода")
        print("0. Выход")
        choice = input("Выбор: ").strip()
        if choice == '1':
            run_hamming()
        elif choice == '2':
            run_iterative()
        elif choice == '3':
            run_analysis()
        elif choice == '0':
            print("Выход.")
            break
        else:
            print("Неверный выбор.")
if __name__ == "__main__":
    main()

```

```

=====
ЛАБА 2. ПОМЕХОУСТОЙЧИВОЕ КОДИРОВАНИЕ
=====
1. Часть 1: Код Хемминга
2. Часть 2: Итеративный код
3. Анализ корректирующей способности итеративного кода
0. Выход
Выбор: 2

=====
ЛАБА 2, ЧАСТЬ 2: ИТЕРАТИВНЫЙ КОД
=====

Примеры k1 x k2: 4x4, 4x5, 4x6, 4x8, 5x8
Введите k1 (строк, >= 2): 2
Введите k2 (столбцов, >= 2): 4
Число групп паритетов (2, 3 или 4): 2

Параметры кода: k = 8, r = 6, n = 14, групп паритетов = 2
Скорость кода R = k/n = 8/14 = 0.571

Ввод информационного сообщения:
1. Текст (ASCII)
2. Двоичная строка
3. Файл
Выбор: 1
Введите текст: привет
Получено бит: 66
Биты: 100001111111000100000010000111000100001100101000011010110001000010

```

```

Сообщение разбито на 9 блок(ов) по 8 бит.

-----
Блок #1
-----
Xk          : 10000111
Xn          : 10000111111111
Xr          : 111111
Матрица k1 x k2:
  1000
  0111
Число ошибок: 2
Ошибки внесены в позиции: [1, 12]
Yn          : 11000111111101

Строки с ошибкой паритета : [0]
Столбцы с ошибкой паритета: [1, 2]
Найденные позиции ошибок  : [(0, 1), (0, 2)]
Исправленное Yk'          : 10100111
Статус                    : detected
Xk верно                   : False

-----
Блок #2
-----
Xk          : 11110001
Xn          : 11110001011110
Xr          : 011110
Матрица k1 x k2:
  1111
  0001

```

Вывод: в ходе лабораторной работы я приобрела практические навыки кодирования и декодирования двоичных данных при использовании кода Хемминга и при использовании итеративных кодов.